

Reviewer Pack — Tropicalized Geometric Quantization Rank vs BP_ReadTwice Cir...

Entry 202f24baf808 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 08:24:00 UTC

Tropicalized Geometric Quantization Rank vs BP_ReadTwice Circuit Threshold

Entry ID: 202f24baf808 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 08:24:00 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization (Tropicalization) **Field B** (complexity object): Complexity Theory: BP_ReadTwice Circuit Threshold

Statement:

[‘For any given Boolean function f , the tropicalized geometric quantization rank $\rho(f)$ is $O(\log(n))$ for all read-twice branching programs P computing f with size $|P|$.’, ‘However, if f requires a circuit of size $|C|$ to be computed by an unrestricted BP, then $\rho(f) = \Omega(|C|)$.’, ‘The difference in $\rho(f)$ between circuits and BPs is at least logarithmic.’]

Rationale (proposer’s reasoning):

[‘Geometric quantization provides a bridge between classical geometry and quantum mechanics, offering a way to encode complex structures into geometric objects. Tropicalization simplifies these objects, potentially revealing hidden structure that can be exploited in complexity theory.’, ‘The proposed conjecture suggests that the quantization rank of a function could serve as an invariant for BP threshold, which has not been explored before in this context.’, ‘If true, this would mean that geometric quantization could lead to new techniques for proving lower bounds on BP complexity.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): abcaa5d480f3af02

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The tropicalized geometric quantization rank $\rho(f)$ for read-twice BPs is $O(\log(n))$ and at least logarithmic higher than BP circuits.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)
        return clauses

    def compute_quantization_rank(cnf):
        # Simplified procedure to compute the Grothendieck-Witt class modulo 2
        rank = 0
        for clause in cnf:
            rank += len([x for x in clause if x != 0])
        return rank % 2

    def generate_bp(cnf):
        # Placeholder for BP generation logic
        return random.randint(1, 10)

    def generate_circuit(cnf):
        # Placeholder for circuit generation logic
        return random.randint(1, 10)

    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = generate_cnf(n)
    rho_f = compute_quantization_rank(cnf)
    bp_size = generate_bp(cnf)
    circuit_size = generate_circuit(cnf)
```

```

return {
    "metric_name": "Quantization Rank vs BP/Circuit Size",
    "metric_value": abs(bp_size - circuit_size),
    "instances_tested": 1,
    "conjecture_holds": rho_f <= math.log(n) and rho_f >= math.log(circuit_size),
    "counterexample": "" if rho_f <= math.log(n) and rho_f >= math.log(circuit_size) else f"rho_f < math.log(n) and rho_f > math.log(circuit_size)"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

.302585092994046, circuit_size=8'}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 8, 'instances_tested': 1}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 1, 'instances_tested': 1}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 3, 'instances_tested': 1}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 1, 'instances_tested': 1}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 7, 'instances_tested': 1}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 6, 'instances_tested': 1}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 5, 'instances_tested': 1}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 2, 'instances_tested': 1}
TRIAL: {'metric_name': 'Quantization Rank vs BP/Circuit Size', 'metric_value': 7, 'instances_tested': 1}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b5162199.py", line 71, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]))

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15657
2	preregistration	ollama_remote	glm4:latest	0	0	9160
3	novelty	ollama_remote	glm4:latest	0	0	8685
4	novelty	ollama_remote	glm4:latest	0	0	8206
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16281
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12267
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12035
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7685
9	judge	ollama_remote	glm4:latest	0	0	11704

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 101682 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/202f24baf808.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/202f24baf808.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/202f24baf808.tar.gz (if generated)